

Nemotron-Labs-Diffusion

A Tri-Mode Language Model Unifying Autoregressive, Diffusion, and Self-Speculation Decoding

AR을 버리지 않고, diffusion을 병렬 초안과 생성 모드로 재 사용한다

01 AUTOREGRESSIVE

causal attention

02 DIFFUSION

block-bidirectional

03 SELF-SPECULATION

draft + verify

이 논문의 한 문장 결론

AR과 diffusion은 경쟁 관계가 아니라,
서로의 약점을 보완하는 두 개의 '동작 모드'다.

Q1 둘을 합칠 수 있는가? → 공동 loss와 attention 전환으로 통합

Q2 MTP보다 빠를 수 있는가? → diffusion draft + AR verify로 높은 수용 길이

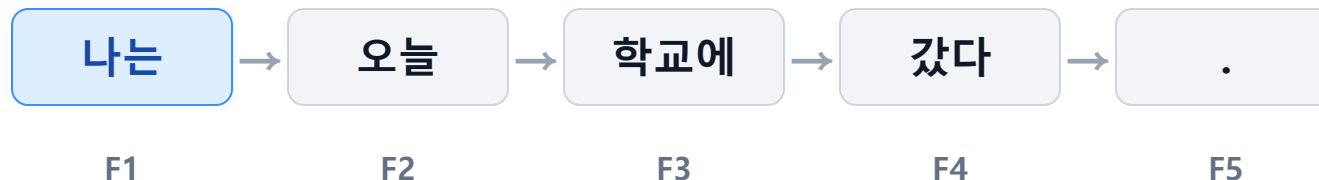
Q3 diffusion의 상한은 충분한가? → SOL 기준 실용 구현보다 큰 병렬성 여지

출발점: AR은 한 토큰씩 기다린다

1

token / forward

다음 토큰은 앞 토큰이
확정돼야 계산할 수 있다



저동시성에서는 메모리 대역폭이 병목

가중치를 매번 읽지만 한 번에 토큰 하나만 확정 → GPU 계산 자원이 충분히 채워지지 않음

텍스트 diffusion은 '가우시안 노이즈'가 아니다

IMAGE

Continuous diffusion

실수 latent에 Gaussian noise를 더하고
연속값을 조금씩 복원

$$\mathbf{z}_t = \sqrt{\alpha_t} \mathbf{z}_0 + \sqrt{(1-\alpha_t)} \boldsymbol{\varepsilon}$$

0.13 → 0.18 → 0.25 → 0.37

TEXT

Masked discrete diffusion

이산 token ID를 [MASK] 상태로 바꾸고
원래 범주를 반복적으로 분류

[MASK] → 'Seoul'

BERT식 복원을 생성 루프로 확장

공통점은 '오염 → 반복 복원'이고, 오염의 형태는 완전히 다르다.

생성 순서가 토큰 위치 순서와 분리된다

AR

위치 순서 = 확정 순서

나는	[]	[]	[]	[]
나는	오늘	[]	[]	[]
나는	오늘	학교에	갔다	.

DIFFUSION

위치는 고정, 확정 순서는 자유

[MASK]	[MASK]	[MASK]	[MASK]	[MASK]
[MASK]	오늘	[MASK]	[MASK]	.
나는	오늘	[MASK]	갔다	.
나는	오늘	학교에	갔다	.

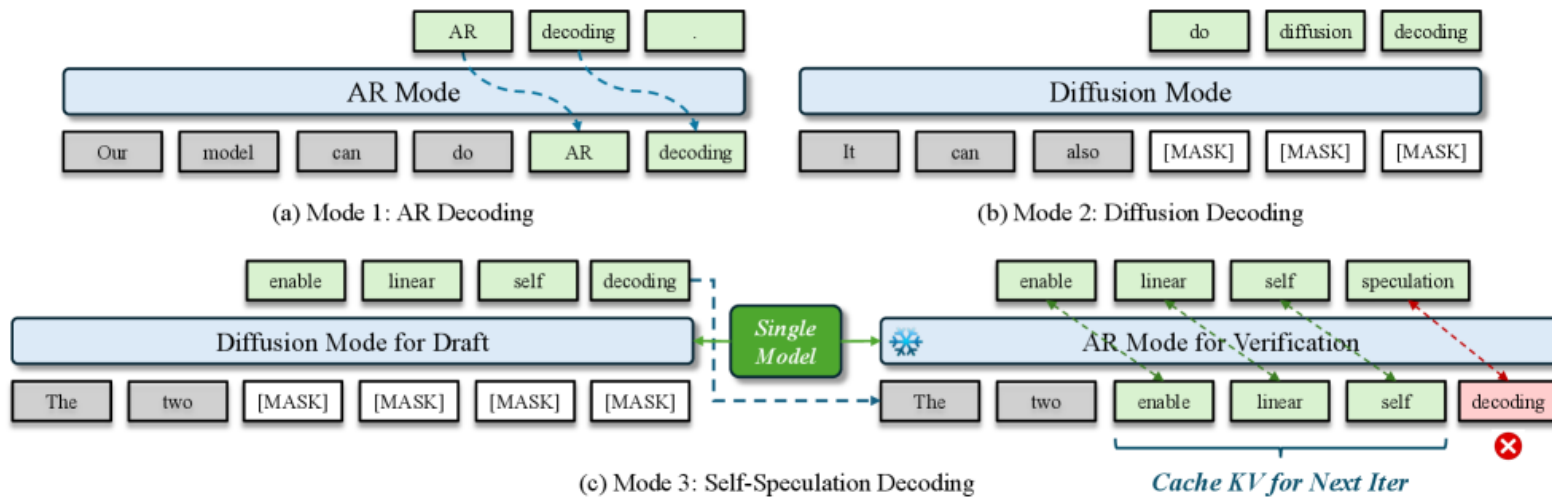
한 forward의 새 예측끼리 즉시 소통하는 것은 아니다. 일부를 확정 한 뒤 다음 forward에서 문맥이 된다.

블록 안은 양방향, 블록 사이는 causal



블록 생성: $1 \rightarrow 2 \rightarrow 3$ | 블록 내부 복원: confidence가 높은 위치부터 | 이전 블록 KV cache 재사용

백본은 하나, attention만 바꾼다



동일한 θ

causal mask
→ AR

block-bidirectional
→ Diffusion

두 mask를 연속 사용
→ Self-speculation

학습 목표는 AR + diffusion

AR OBJECTIVE

$$\mathcal{L}_{\text{AR}}(\theta) = \mathbb{E}_{\mathbf{x}} [- \sum_i \log p_{\theta}(\mathbf{x}_i \mid \mathbf{x}_{<i})]$$

깨끗한 문장에서 다음 토큰 예측 → 왼쪽에서 오른쪽이라는 언어 prior 유지

DIFFUSION OBJECTIVE

$$\mathcal{L}_{\text{diff}}(\theta) = \mathbb{E}_{\mathbf{t}, \mathbf{q}} [- 1/t \cdot \sum_{\beta} \log p_{\theta}(\mathbf{x}^{\beta} \mid \mathbf{x}_t^{\beta}, \mathbf{x}_{<^{\beta}})]$$

clean prefix에 조건화해 현재 noisy block을 복원 → 미래 토큰을 함께 고려

$$\mathcal{L}(\theta) = \mathcal{L}_{\text{AR}}(\theta) + 0.3 \cdot \mathcal{L}_{\text{diff}}(\theta)$$

학습 MASK와 추론 sampler는 다르다

TRAINING

$t \sim \text{Uniform}[0,1]$

noise level에 비례해 현재 블록의 무작위 위치를 MASK 처리

t=0.2	나는	[M]	학교 에	갔다	.
t=0.5	[M]	오늘	[M]	갔다	[M]
t=0.8	[M]	[M]	[M]	갔다	[M]

INFERENCE

Sampler = acceptance policy

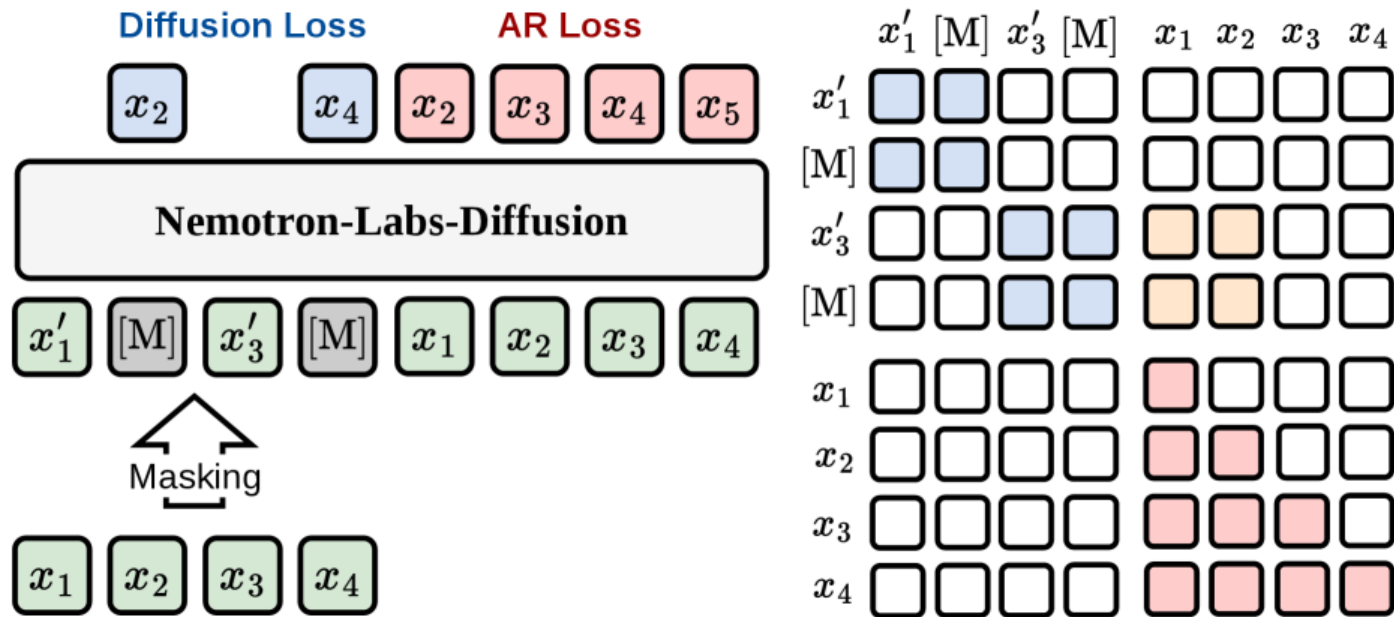
현재 MASK 중 어떤 top-1 예측을 지금 확정할지 선택

다음 MASK 집합 = 현재 MASK

– 이번 단계에서 확정된 위치

sampler가 반복 횟수를 직접 출력하지는 않는다

Dual-stream⁰이 두 loss를 한 번에 계산한다



Noisy stream

- 블록 내부 bidirectional
- 이전 clean prefix 참조
- diffusion loss 계산

Clean stream

- strictly causal
- 미래 label leakage 차단
- AR loss 계산

같은 forward-backward에서 두 objective가 동일한 θ 를 업데이트

강한 AR에서 시작해 diffusion을 켜다

STAGE 1

Pure AR

1T tokens

$\alpha = 0$

256

NVIDIA H100 GPUs

STAGE 2

Joint AR + Diff.

300B tokens

$\alpha = 0.3$

4,096

pretraining sequence

SFT

Joint instruction tuning

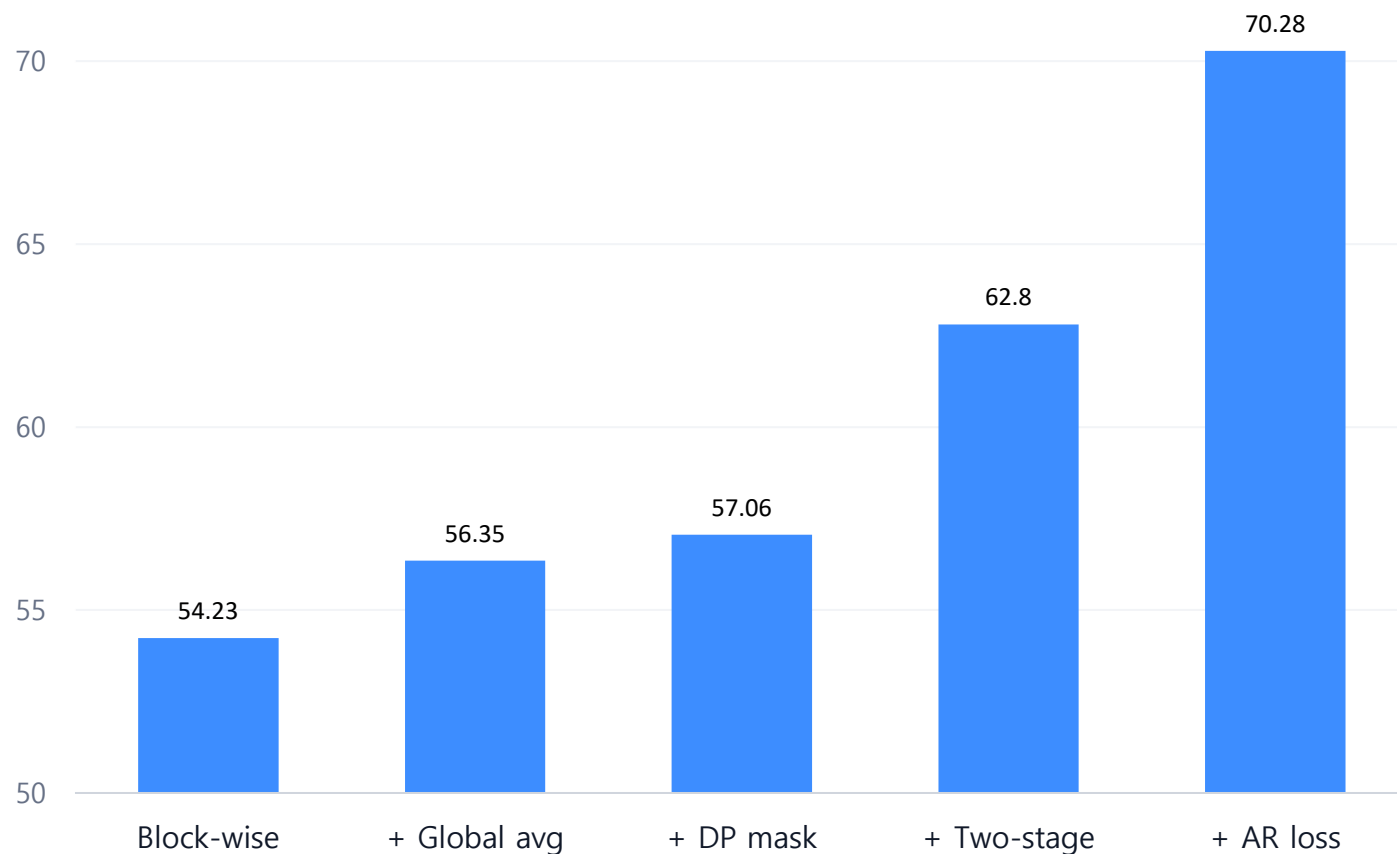
45B tokens

answer-only loss

16K

SFT sequence length

성능을 만든 핵심은 AR prior였다



+16.05%p

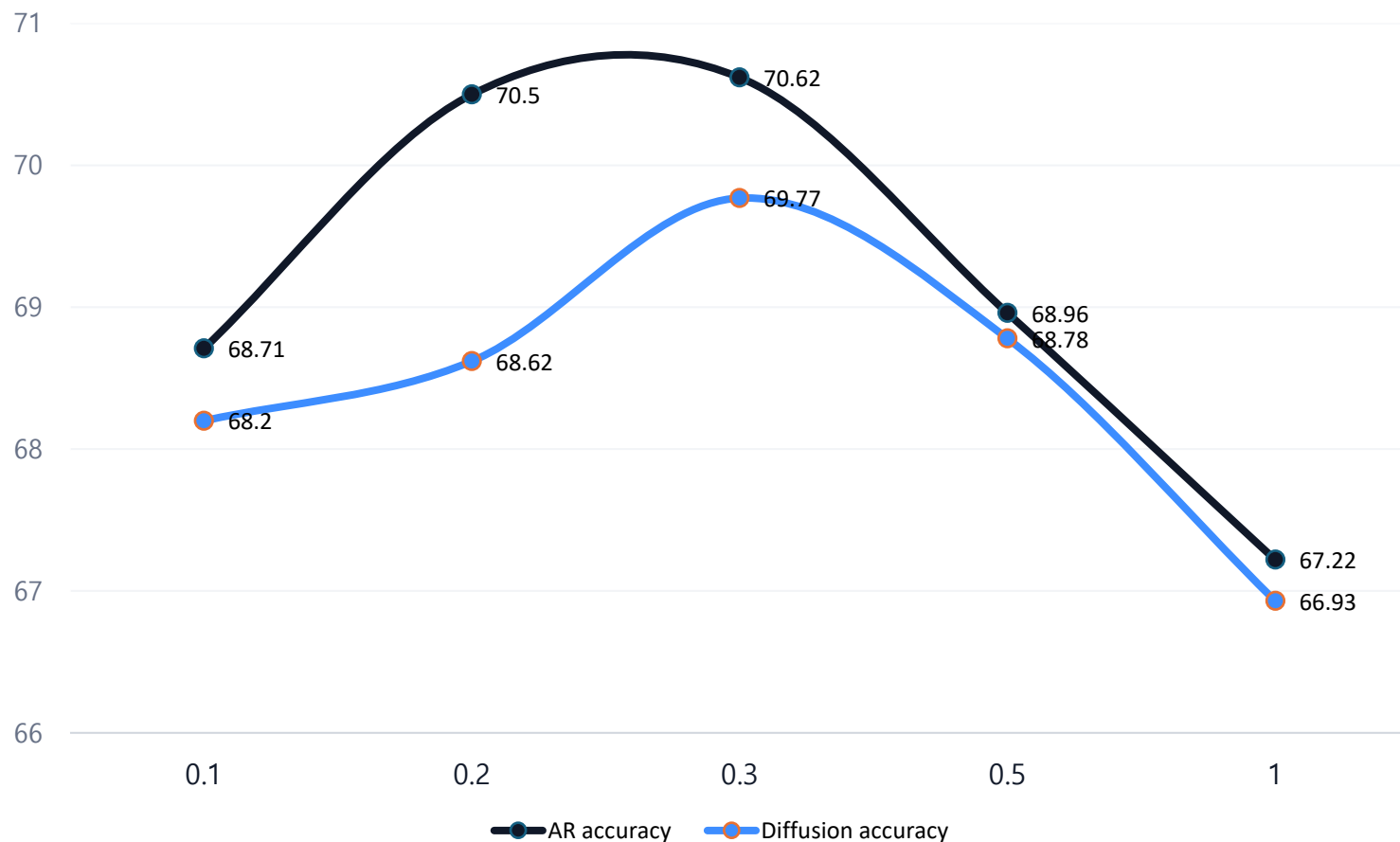
전체 파이프라인 개선

+7.48%p

마지막 AR loss의 단일 기여

두 단계 AR 초기화도 +5.74%p

$\alpha=0.3$ 에서 두 모드가 함께 좋아졌다



0.3

공동 최적점

두 모드가 서로 희생하지 않고 함께 peak

+0.43%p

Instruct AR 평균

diffusion loss 추가 후 오히려 소폭 상승

Mode 1 — AR: 고통시성의 안전한 기본값

$$p_{\theta}(x_i \mid x_{< i})$$

causal attention · 1 token/forward



언제 쓰나

- 높은 동시 사용자 수
- 큰 batch로 GPU가 이미 compute-bound
- 기존 AR serving pipeline 유지 필요
- 가장 보수적인 품질 기준이 중요

Tri-mode 모델은 AR 성능을 포기하지 않기 때문에 기존 LLM의 drop-in 대체가 가능하다.

Mode 2 — 여러 MASK를 병렬 확정한다



반복 루프

- ① 모든 MASK 예측
- ② sampler가 위치 선택
- ③ 선택 위치 commit
- ④ 남은 MASK 재 예측

$$\text{TPF} = \text{block size} / \text{forward 횟수}$$

sampler는 '무엇'보다 '언제'를 학습한다

BACKBONE

이 MASK에 무엇이 들어가는가?

나는 오늘 회사에 갔다

confidence: 0.91 / 0.93 / 0.89 / 0.92

SAMPLER

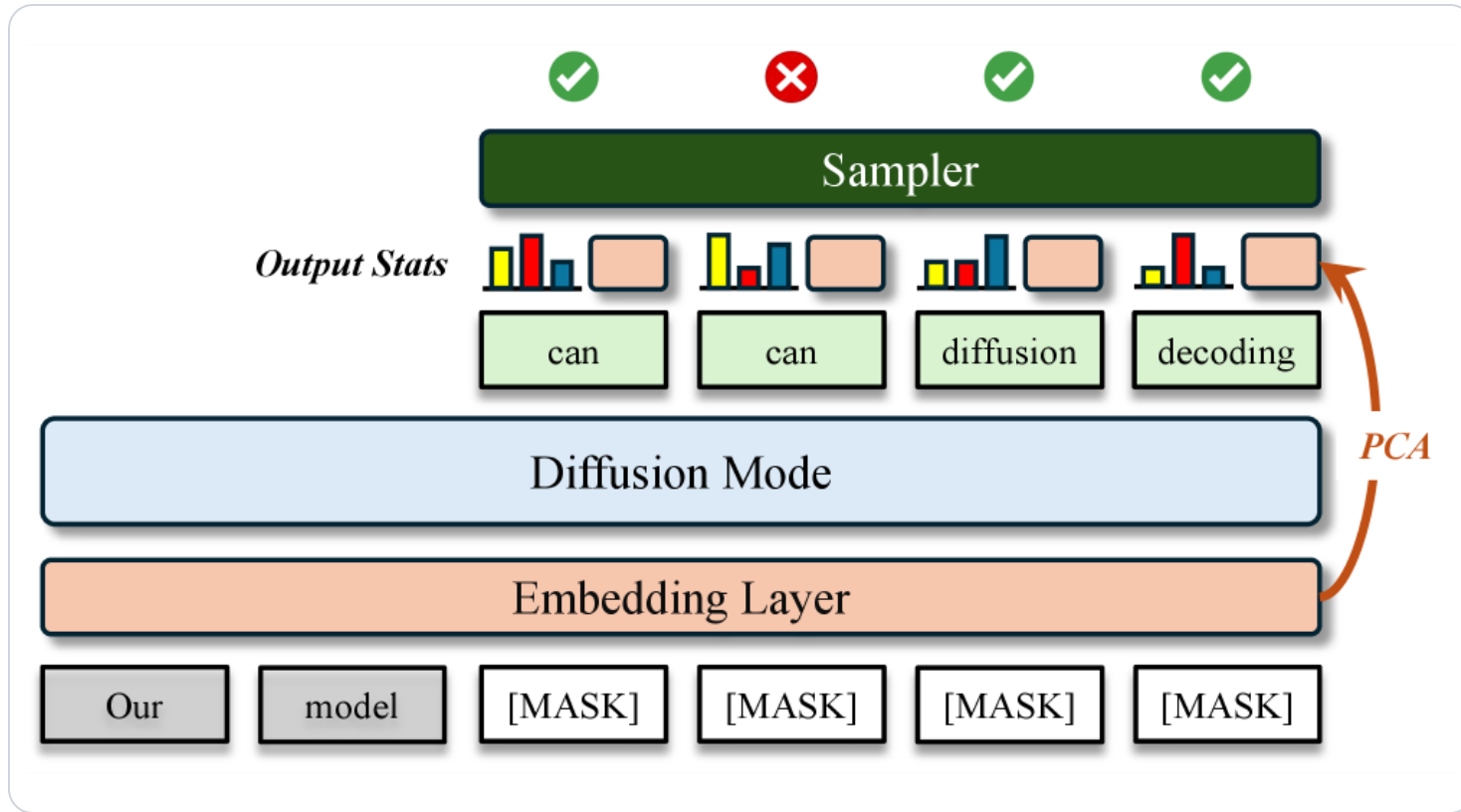
지금 확정해도 끝까지 유지되는가?

✓ ✓ ✗ ✓

'회사에'는 확률이 높지만
문맥이 채워지면 '학교에'로 바뀔 수 있음

높은 confidence ≠ 최종 안정성 | 여러 위치를 동시에 commit하면 서로의 조건부 분포가 바뀐다

4.8M sampler가 블록 전체를 함께 본다



4 layers

bidirectional Transformer

4.8M

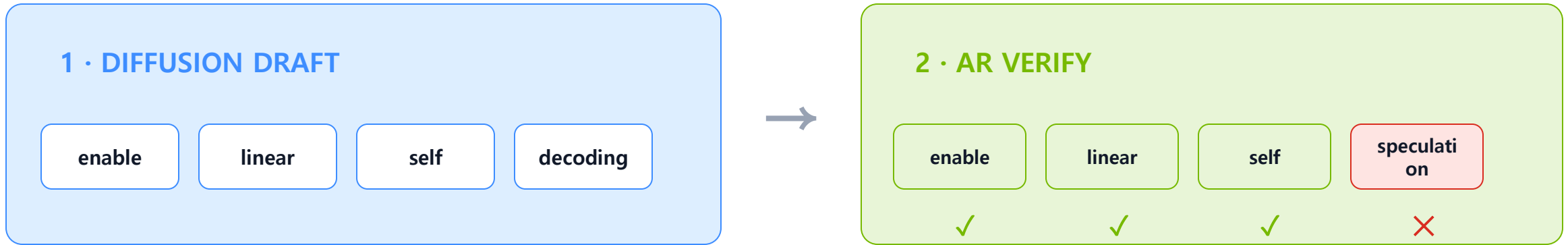
8B backbone의 약 0.06%

20M

denoising trajectories

144-D features: top-1 확률 · margin · top-3 mass · entropy · top-3 semantic embeddings

Mode 3 — 같은 모델이 draft하고 verify한다



longest matching prefix

enable linear self

+ first rejected position의 AR token

speculation

두 번의 forward로 1 ~ k+1개 토큰 확정 · prefix KV cache 공유

Linear SS가 현재의 실용적 승자다

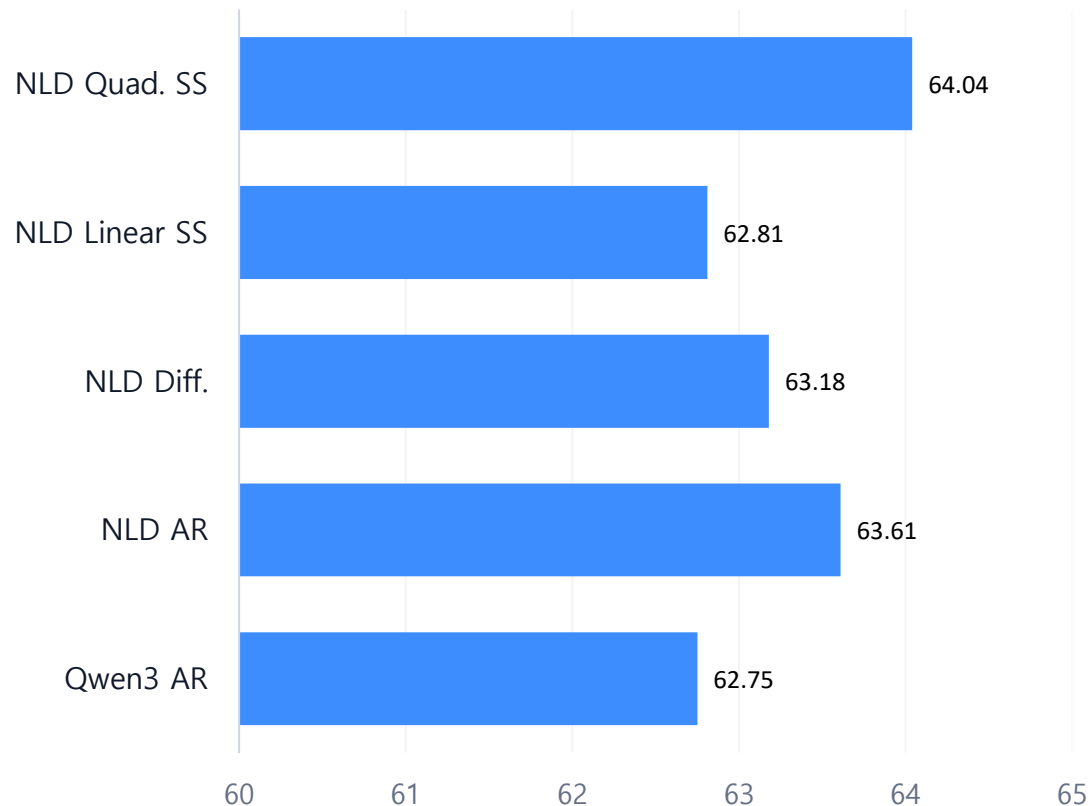
구분	Linear SS	Quadratic SS
forward 구조	diffusion draft + AR verify	한 forward에서 draft·verify 병렬
수용 방식	일치하는 prefix	k^2 mask로 모든 분기 준비
8B 평균 TPF	5.99× (LoRA)	6.38×
실장 상태	최적화된 기본 경로	FlexAttention kernel 부담
논문 결론	현재 real-device 효율 우세	인프라 개선 시 잠재력

+32.5% 8B LoRA 정렬의 TPF 개선

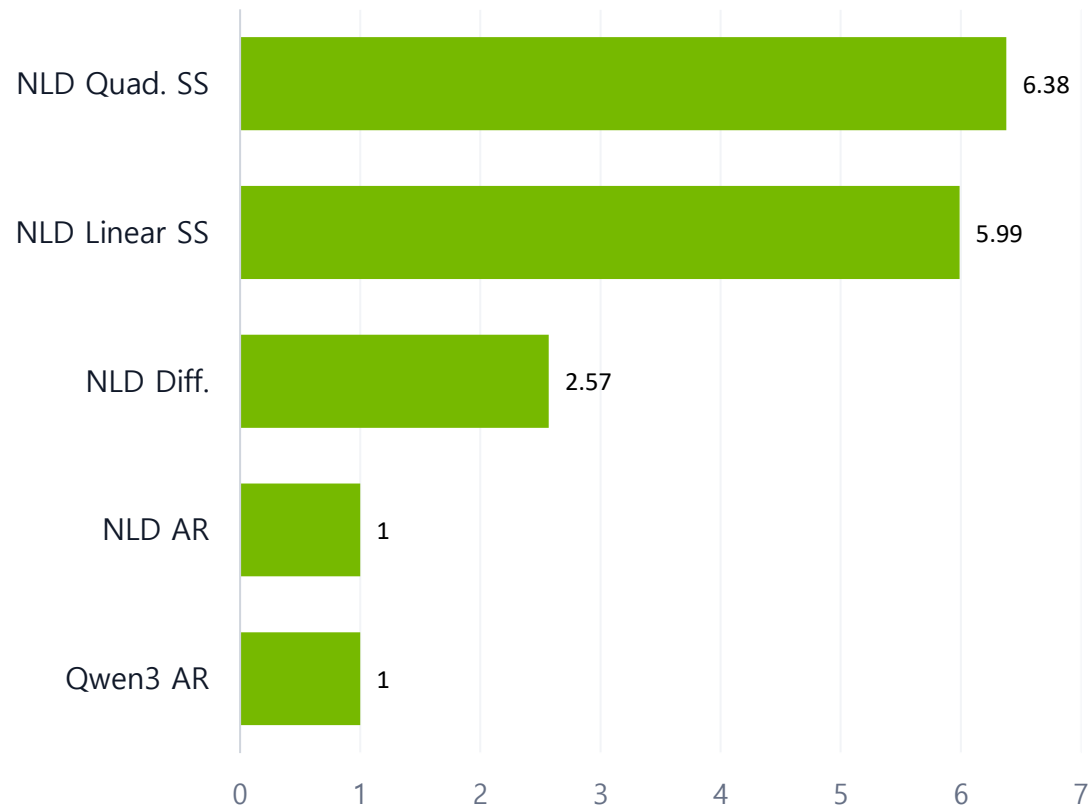
36M drafter-only LoRA 파라미터

8B에서 정확도는 유지하고 TPF는 6배로

평균 정확도 (%)



Tokens per forward



동일한 10개 benchmark 평균 · non-thinking mode

실제 하드웨어에서도 저동시성 이득이 컸다

3.3×

GB200 vs NLD AR

batch size 1 · linear self-speculation

1015

tok/sec

optimized kernel 사용 시 약 3.97×

2.4×

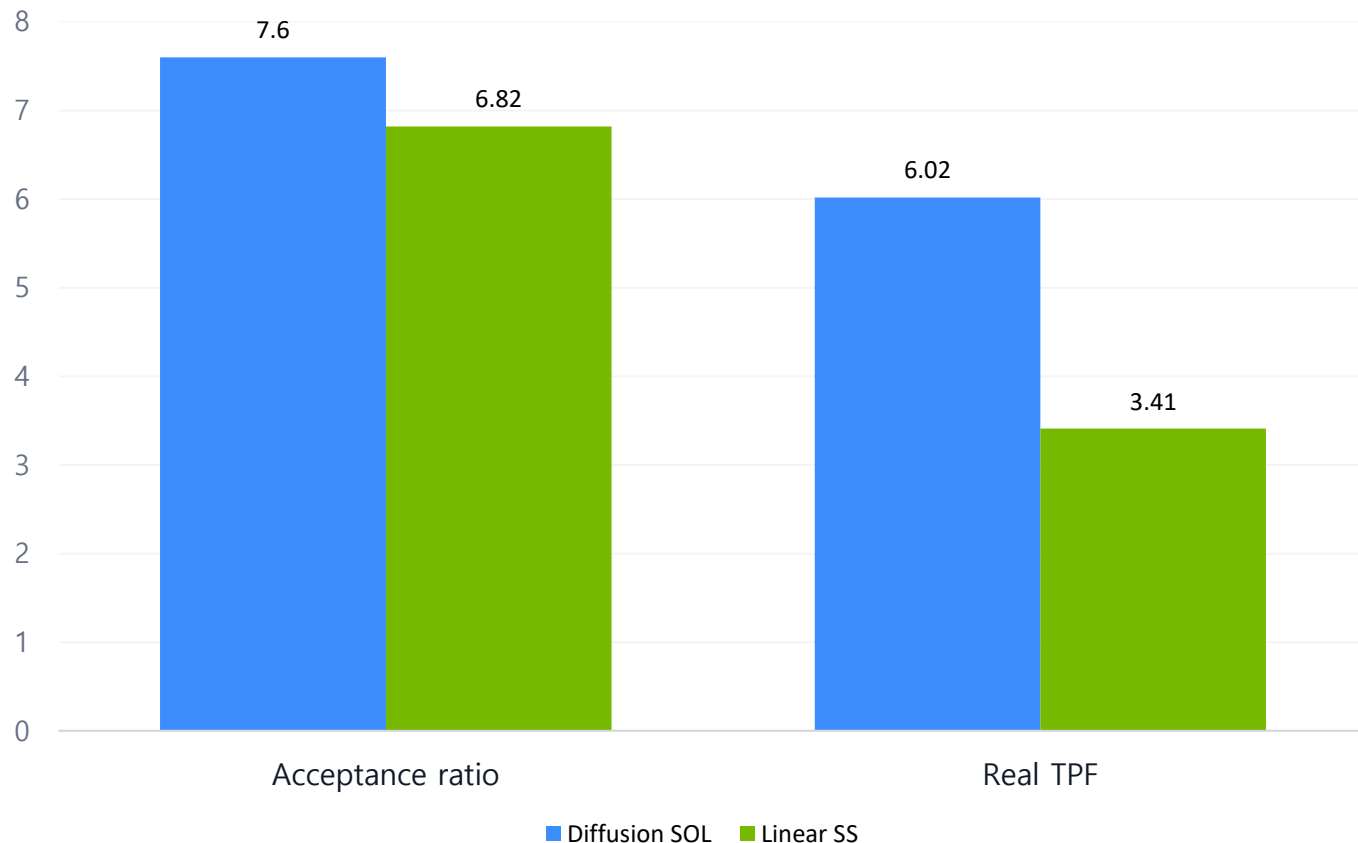
GB200 vs Eagle3

동일한 low-concurrency 비교

중요한 조건

효과는 batch size, concurrency, precision, kernel, block length에 민감하다.
고동시성에서는 AR이 시스템 처리량 관점에서 다시 유리할 수 있다.

SOL 분석: 현재 sampler는 상한에 못 미친다



+76.5%

real TPF 잠재 이득

6.02 vs 3.41

7.60×

diffusion SOL 평균

block size 32 · oracle safe subset

SOL은 실제 알고리즘이 아니라 oracle 상한 분석이다

이 논문의 강점은 '통합'과 '전환 가능성'

- | | | |
|----|---------------------|---|
| 01 | 한 checkpoint, 세 운영점 | 모델 교체 없이 concurrency에 맞춰 decoding mode 전환 |
| 02 | joint training을 실증 | AR prior가 diffusion 정확도를 높인다는 ablation 제공 |
| 03 | MTP와 시스템 단위 비교 | TPF가 아닌 실제 GPU throughput과 acceptance length 측정 |
| 04 | 상한과 현재 구현을 분리 | SOL 분석으로 sampler 연구의 남은 공간을 정량화 |

다만 속도 주장은 조건과 구현에 민감하다

1

Benchmark comparability

AR baseline은 공통 프레임워크, diffusion baseline은 공식 파이프라인 사용 → 완전 동일 조건은 아님

2

Sampler generalization

20M trajectory가 math·code·STEM·chat 중심 → 다른 도메인과 긴 문맥에서 재검증 필요

3

Oracle vs deployable

SOL의 safe subset 탐색은 실제 inference에서 쓸 수 없는 상한

4

System dependence

quadratic SS가 TPF는 높아도 kernel 미성숙으로 실제 속도는 낮음

Reviewer's take: '6× TPF'와 '4× throughput'은 같은 주장이 아니며, 배포 조건을 함께 읽어야 한다.

AR을 버린 것이 아니라, AR을 diffusion의 기준으로 재사용했다.

01

Training

$\mathcal{L}_{AR} + 0.3\mathcal{L}_{diff}$ 로 하나의 training-mode backbone

02

Inference

diffusion이 병렬 예측하고 AR이 검증

03

Deployment

저동시성은 self-speculation,
고동시성은 AR

04

Open problem

SOL에 가까운 non-prefix sampler

토론 질문

① sampler를 end-to-end로 학습할 수 있을까? ② 문단 단위 병렬성까지 확장할 수 있을까?